

WEEK 04 INTERNSHIP REPORT

Full Stack Development Internship
UpSkill Campus × UniConverge Technologies (UCT)

Intern Name:	Arsh Kamal
Internship Track:	Full Stack Web Development
Organization:	UpSkill Campus x UniConverge Technologies (UCT)
Report Period:	Week 04

1. Internship Overview

This report covers the fourth week of my Full Stack Development Internship, undertaken under UpSkill Campus in collaboration with UniConverge Technologies (UCT). The internship is designed to build practical and conceptual knowledge of full stack web development, starting with the fundamentals of front-end and back-end technologies. During Week 04, the training material focused on introducing the concept of full stack development and the basics of HTML, which form the foundation for building web pages.

2. Week 4 Learning Overview

The Week 04 session began with an introduction to what a "full stack" means in web development, followed by an explanation of the two main areas of a full stack developer's skill set: front-end development and back-end development. The material then introduced the MEAN stack (MongoDB, Express, AngularJS, Node.js) as an example of a full JavaScript-based technology stack, along with the software components required for full stack development. The second half of the session covered the basics of HTML, including HTML elements, tags, attributes, and the standard structure of an HTML document, supported by simple code examples and a hands-on exercise.

3. Topics Covered

- Introduction to Full Stack Web Development
- Front-End Development
- Back-End Development
- Frameworks (front-end and back-end)
- The MEAN Stack (MongoDB, Express, AngularJS, Node.js)
- Installing Software Components (Node.js, NPM, Code Editor, MongoDB, Browser)
- Basics of HTML
- HTML Tags: Elements, Tags, and Attributes
- HTML Document Structure
- Code Validation
- Example: Creating a Basic HTML Page

4. Detailed Learning

4.1 What is a Full Stack?

A "stack" refers to all the components required to run an application, including internal elements such as UI frameworks, MVC frameworks, and data access libraries, as well as external elements such as the operating system, application server, and database. A full stack web developer is someone who has general knowledge across a wide range of technologies, along with in-depth specialization in a few of them.

4.2 Front-End Development

Front-end development deals with the presentation of a website, that is, how information is displayed in browsers and on mobile devices. Front-end developers primarily work with HTML, CSS, and JavaScript to control layout, text, images, colors, fonts, and buttons. The main goal of a front-end developer is to build the interface through which users interact with an application. This role can also extend into user experience (UX) and user interface (UI) design.

4.3 Back-End Development

Back-end development involves creating, editing, updating, and retrieving data. Common back-end scripting languages include JavaScript, PHP, Ruby, and Python. Back-end developers write the logic to receive information from the user and store it in a database. Databases are of two main types: relational databases (such as Oracle, SQL Server, PostgreSQL, and MySQL) and non-relational databases (such as MongoDB, Oracle, and Couchbase). SQL is the language used to interact with relational databases. Back-end development also involves server management, either through self-managed servers or cloud platforms such as Heroku or Amazon Web Services (AWS).

4.4 Frameworks

Frameworks help developers avoid writing repetitive, proprietary code by providing tested and reusable solutions. On the front-end, libraries such as jQuery and frameworks such as AngularJS and EmberJS simplify common development tasks. On the back-end, popular frameworks include Express (for Node.js), Rails (for Ruby), Django (for Python), and CakePHP (for PHP). Frameworks make development easier by offering a set of conventions for displaying, storing, and accessing data.

4.5 The MEAN Stack

MEAN is a full JavaScript technology stack made up of MongoDB, Express, AngularJS, and Node.js:

- **Node.js** – A server-side JavaScript execution environment built on Google Chrome's V8 JavaScript runtime, used to build scalable and concurrent applications.
- **Express** – A lightweight Node.js framework used to build single-page and multi-page web applications, inspired by the Ruby framework Sinatra.
- **MongoDB** – A schema-less NoSQL database that stores data in binary JSON format, making it easy to exchange data between client and server.
- **AngularJS** – A JavaScript framework developed by Google, known for features such as two-way data binding, used for front-end development.

4.6 Installing Software Components

The material also explained the basic software setup required for full stack development:

- **Node.js** – Installed by downloading and unzipping the files, then updating the path system variable.
- **NPM (Node Package Manager)** – Installed automatically along with Node.js.

- **Code Editor** – A plain text editor (such as Notepad++) or an Integrated Development Environment (IDE) such as Atom or Visual Studio.
- **MongoDB** – Can be used as a free cloud instance through MongoDB Atlas, or installed locally.
- **Browser** – Any modern browser such as Chrome, Firefox, or Edge.

4.7 Basics of HTML

HTML (HyperText Markup Language) gives content structure and meaning by defining it as headings, paragraphs, images, and so on. CSS (Cascading Style Sheets) is used to style the appearance of that content. HTML and CSS are kept independent of each other: HTML represents content, while CSS represents appearance.

4.8 HTML Elements, Tags, and Attributes

- **Elements** – Designators that define the structure and content of a page, such as headings (h1 to h6), paragraphs (p), and others like a, div, span, strong, and em.
- **Tags** – Created using angle brackets around an element name. Tags usually occur in pairs: an opening tag (e.g. <div>) and a closing tag (e.g. </div>). The content between the two tags is the element's content.
- **Attributes** – Properties that provide additional information about an element, such as id, class, src, and href. Attributes are written inside the opening tag as attribute_name="value".

4.9 HTML Document Structure

Every HTML document follows a required structure consisting of the <!DOCTYPE html> declaration, followed by the <html> element, which contains the <head> and <body> elements. The <head> element holds metadata such as the character encoding and page title, which is not displayed on the page itself. The <body> element contains all the visible content of the web page. Nested elements, such as <head> and <body> inside <html>, should be indented to keep the code organized and legible.

4.10 Self-Closing Elements and Code Validation

Not all elements require both opening and closing tags. Self-closing (void) elements, such as
, , <input>, <link>, and <meta>, receive their content or behavior from attributes within a single tag. The material also introduced code validation using the W3C HTML and CSS validators, which help check code for mistakes and ensure it renders correctly across browsers.

5. Activities / Practical Learning

As part of Week 04, the following practical elements were included in the training material:

- Reviewing a code example of a basic HTML page structure using <!DOCTYPE html>, <html>, <head>, and <body> tags.
- Studying an example (Example 1-1: Basic HTML Page) that demonstrated creating a file named Example1_1.html with a heading and a paragraph, saving it, and running it in a browser.
- Attempting the "Test Your Knowledge" quiz for the Application Development module on the UpSkill Campus platform, covering the concepts taught during Week 04.

In this attempt, I scored 20 out of 20 marks (100% accuracy) across all 10 questions in the quiz, with no incorrect or unanswered questions, completed within the given time.

6. Key Learning Outcomes

- Understood the meaning of a "full stack" and the role of a full stack developer.
- Learned the distinction between front-end and back-end development and the technologies associated with each.
- Understood the purpose of frameworks in simplifying front-end and back-end development.
- Learned about the MEAN stack and the role of MongoDB, Express, AngularJS, and Node.js within it.
- Understood the basic software setup required to start full stack development.
- Learned the fundamentals of HTML, including elements, tags, attributes, and document structure.
- Understood the difference between paired tags and self-closing (void) elements.
- Learned the importance of code validation for writing clean and browser-compatible code.

7. Challenges & Understanding

During Week 04, one of the initial challenges was getting used to the number of new terms introduced together, such as stack, framework, and the different components of the MEAN stack, and understanding how they relate to one another. Differentiating clearly between the roles of front-end and back-end development, and understanding how frameworks fit into both, required careful reading of the material. While studying HTML basics, understanding the difference between elements, tags, and attributes, as well as identifying which HTML elements are self-closing, needed a bit of repetition to become clear. Working through the concepts step by step, along with the provided code examples, helped in building a clearer understanding of these topics.

8. Skills/Knowledge Gained

- Conceptual understanding of full stack web development.
- Awareness of front-end technologies: HTML, CSS, and JavaScript.
- Awareness of back-end technologies: server-side languages, relational and non-relational databases, and server management.
- Basic knowledge of the MEAN stack (MongoDB, Express, AngularJS, Node.js).
- Familiarity with setting up a basic full stack development environment.
- Ability to identify and use basic HTML elements, tags, and attributes.
- Understanding of the standard structure of an HTML document.
- Awareness of code validation tools for HTML and CSS.

9. Conclusion

Week 04 of the Full Stack Development Internship provided a solid conceptual foundation for the rest of the program by introducing the core idea of full stack development along with the basics of HTML. Understanding the distinction between front-end and back-end development, the role of frameworks, and the components of the MEAN stack has given me a clearer picture of how a full stack application is built. Learning the fundamentals of HTML, including its structure, elements, tags, and attributes, has laid the groundwork for building web pages in the upcoming weeks. Overall, this week's learning has strengthened my basic understanding and prepared me to move ahead confidently into more detailed front-end and back-end topics in the coming weeks of the internship.

— End of Week 04 Report —